

Group Project Report

MOSAIC

A Multi-Agent System for Hyper-Personalised Email Marketing

Team	Team 13
Members	Ong Qing Quan (OQQ) — engineering lead: Orchestrator, Segmentation & Insight, Email Composer & Assembly, Campaign Console, CI and versioning Agustinus Benyamin Prasetyo (ABP) — domain and assurance lead: Creative Direction, Compliance & Brand Guardian, Responsible AI instrumentation, AI security test corpus
Project sponsor	None — self-initiated project. All customer data is synthetic and generated by the team.
Repository	github.com/bearly-profitable/MOSAIC
Submitted	October 2026

Contents

1. Executive Summary 4
1.1 Objective and scope 4
1.2 Key highlights 4
1.3 Constraints and assumptions 4
2. System Overview 6
2.1 The business problem 6
2.2 Why a multi-agent architecture 6
2.3 Stakeholders 7
2.4 How the agents work together 7
2.5 How the benefits are demonstrated 8
3. System Architecture 9
3.1 Logical architecture 9
3.2 Physical architecture 10
3.3 Deployment strategy 11
3.4 Agent communication protocols 11
3.5 Data architecture 13
3.6 Justification of architectural styles and technology choices 14
3.7 Scalability 14
4. Agent Roles and Design 16
4.1 Orchestrator 16
4.2 Segmentation & Insight Agent 16
4.3 Creative Direction Agent 17
4.4 Email Composer & Assembly Agent 18
4.5 Compliance & Brand Guardian Agent 19
4.6 Evaluator (sidecar) 20
4.7 Shared prompt-engineering patterns and fallbacks 20
5. Explainable and Responsible AI Practices 22
5.1 Alignment across the development and deployment lifecycle 22
5.2 Fairness and bias mitigation 22
5.3 Explainability 23
5.4 Accountability and human oversight 24
5.5 Manipulation guardrail and ethical exclusions 24
5.6 Governance framework alignment 25
5.7 Data minimisation 25
6. AI Security Risk Register 26
6.1 Threat model 26
6.2 AI security testing 27
7. MLSecOps / LLMSecOps Pipeline 28

7.1 Pipeline overview 28
7.2 Automated testing, including AI security tests 28
7.3 Versioning and tracking 29
7.4 Deployment strategy 29
7.5 Monitoring and alerting 29
7.6 Logging and auditability 30
8. Testing and Evaluation Summary 31
8.1 Types of tests performed 31
8.2 Results 31
8.3 Key findings 32
9. Reflection 33
9.1 What went well 33
9.2 What changed from the proposal, and why 33
9.3 Challenges 33
9.4 Limitations and future work 34
9.5 Team contributions 34
Appendix A. API Surface 35
Appendix B. Repository Structure 35
Appendix C. Console Screenshots 36

1. Executive Summary

1.1 Objective and scope

Most retailers run an email campaign as three or four creative variants — new, lapsed, high-value and everyone else — because every extra variant needs its own brief, design pass, copy pass, render test and QA sign-off. Two customers with opposite motivations end up in the same bucket, and "personalisation" usually means inserting a first name into a fixed sentence.

MOSAIC treats campaign production as a generation-and-assurance problem. Four specialised agents, coordinated by an Orchestrator over a shared, typed blackboard, discover micro-segments in the operator's own customer list, write a distinct creative direction and email for each segment, and adjudicate every message against ten pre-flight checks. Then the system stops: a person reviews the result and is the only actor who can send. The agents are **Segmentation & Insight**, **Creative Direction**, **Email Composer & Assembly** and the **Compliance & Brand Guardian**; a sidecar **Evaluator** grades their saved output.

The project is a working demonstrator rather than a production system. It runs end to end on a bundled 1,000-contact synthetic customer list, is deployed as two services on Railway with Supabase as the data store, and dispatches approved emails for real through Resend to an address the operator confirms.

1.2 Key highlights

- **Works on any customer CSV.** The feature schema is inferred per run; protected and consent columns are detected by name and handled specially. Nothing is hardcoded to one dataset.
- **Segments that explain themselves.** HDBSCAN clusters engineered features plus text embeddings; each segment carries its z-scored feature contributions, and the language model is only allowed to name what the numbers already found.
- **Fairness by construction and by test.** Protected attributes never enter the feature matrix; a proxy test then checks whether a segment correlates with them anyway. On the demo data it flagged one of six segments (age, 37% deviation).
- **Grounded generation.** Every model call uses a strict, length-capped JSON schema; code assembles the email layout; offer citations that are not real catalogue rows are dropped.
- **A deterministic Guardian with a veto.** Ten checks across compliance, content and deliverability produce PASS, REVISE or BLOCK. BLOCK is refused server-side, and the checks re-run after every edit. Demo workspace: 5 PASS, 1 REVISE, 0 BLOCK.
- **Traceable by default.** An append-only Decision Ledger records prompt version, model version and artifact lineage for every agent transition; an activity log records every operator action.
- **MLSecOps / LLMSecOps.** GitHub Actions runs a secrets check, an adversarial Guardian corpus and contract pins on every push — 14 of 14 tests pass, and a deliberate negative test proves the gate fails when it should. The eval-v1 sidecar scores the agents at 88.2 overall.

1.3 Constraints and assumptions

Table 1. *Constraints and assumptions*

Area	Constraint or assumption
Team and effort	Two members, guideline effort of 15 person-days each (30 in total), August to October 2026.

Area	Constraint or assumption
Data	All customer data is synthetic and generated by the team (`backend/data/mosaic_customers.csv`, produced by `synthetic_data.py`). The brand and offer catalogue are fictional; the demo workspace is configured as a fictional retailer, “FatCatPillow”.
Models	All model access goes through OpenRouter: `openai/gpt-5.6-luna` for text and judging, `openai/text-embedding-3-small` for embeddings. An image model exists but is never called by a pipeline stage — only when an operator reviews a prompt and presses Generate.
Dispatch	Real, via Resend, but deliberately narrow: each send delivers one rendered email to an operator-confirmed address, with merge tags filled from a real contact in that segment. Bulk delivery to a whole segment is out of scope.
Hardening	Demonstrator, not production-hardened: the API has no authentication or rate limiting, and the backend runs as a single replica because job progress is held in memory (Sections 3.7 and 6).

2. System Overview

2.1 The business problem

A retailer with around 200,000 contacts typically splits a campaign into a handful of variants. The constraint is cost rather than preference: the fifth variant costs about as much to produce as the first, so teams stop at four. A customer who consistently buys on durability and one who hunts discounts both sit inside “high value”, and the campaign speaks to neither. MOSAIC removes that ceiling by generating one creative treatment per discovered micro-segment while moving compliance from after-the-fact sampling to a check on every message.

2.2 Why a multi-agent architecture

We considered two simpler alternatives before choosing a modular, agentic design. Table 2 compares them on the properties the brief asks us to demonstrate.

Table 2. *Alternatives considered*

Criterion	One large prompt	Rules-based platform	MOSAIC (four agents)
New creative per segment	Yes	No — routes copy a human already wrote	Yes
Auditability	Low — one opaque call	High	High — typed artifact and ledger row per stage
Who checks compliance	The generator itself	Rules, on human-written copy	A separate Guardian that can veto any generator
Failure containment	Re-run everything	n/a	Re-run one stage or one email
Cost profile	Model call per recipient — O(n)	Low	Model work per segment — O(k); per-recipient work is deterministic
Right tool per task	One model for everything	No ML	Clustering, schema-bound copywriting and rule-based adjudication each use their own method

The decisive argument is the cost split. Expensive, high-variance work — segment discovery, creative direction and long copy — runs once per segment, O(k). Cheap deterministic work — filling a first name, choosing the cited offer — runs per recipient, O(n). Because n is far larger than k, that split is what makes per-segment creative affordable. The second argument is separation of duties: the agent that vetoes is not the agent that generates, and it is owned by the team member who did not build the generators.

The design has costs we accept. Model spend is real and is metered per stage. Review effort moves rather than disappears, so the console is built to make review fast. And a single off-brand or non-compliant message sent under a client’s name is serious, which is why the Guardian holds a veto and only a person can send.

2.3 Stakeholders

Table 3. *Stakeholders*

Stakeholder	Interest	How MOSAIC serves it
Campaign marketer (primary user)	Ship relevant campaigns quickly without compliance mistakes	Five-tab console; every artifact reviewable and editable; Send is theirs alone
Brand lead	Tone, palette and assets stay on-brand	Brand palette and assets stored once and applied by code; accent colour validated; judge scores brand fit
Data protection officer	Consent and withdrawal under PDPA	Consent column detected at upload; consent re-checked whenever the Guardian runs; synthetic data only
Legal	Claims are substantiated; unsubscribe route present	Offer grounding against the catalogue; unsubscribe check is BLOCK severity
Platform engineering	Cost, reliability, deployability	Cost meter per stage; health-checked containers; CI gate; prompt/model versions in the ledger
Customers (recipients)	Not manipulated, profiled unfairly or exposed	Protected attributes held out and proxy-tested; manipulation guardrails; PII leakage check

2.4 How the agents work together

The console has five tabs, each owning one stage, and each tab is locked until the previous stage has produced a reviewable artifact. The console's route guard and the API enforce the same order independently, so a pasted URL cannot skip a step. Figure 1 shows the flow.

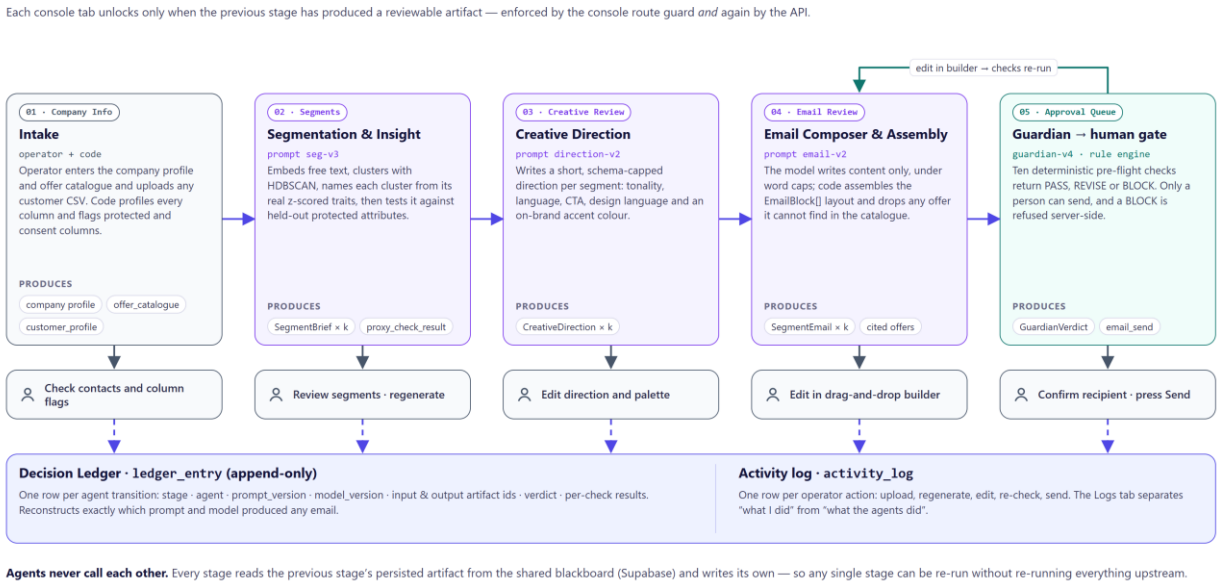


Figure 1. *High-level workflow: five stages, four agents, one human gate, with the Decision Ledger and activity log underneath.*

1. **Company Info (intake).** The operator enters the company profile and offer catalogue and uploads any customer CSV. `csv_ingest.py` profiles every column as numeric, categorical, text, identifier or constant, and flags protected and consent columns.

2. **Segmentation & Insight.** Clusters the audience, names each cluster from its real traits and runs the protected-attribute proxy test.
3. **Creative Direction.** Writes a short, schema-capped direction for each segment.
4. **Email Composer & Assembly.** Writes copy under word caps and assembles a builder-compatible email; the Guardian runs automatically on every composed email.
5. **Approval Queue.** Shows each email exactly as it will render, with its Guardian verdict broken down by check. The operator can edit in the builder — the checks re-run — and finally confirms a recipient and presses Send.

Agents never call each other. Each stage reads the previous stage's persisted artifact from the shared blackboard and writes its own, and every transition is appended to the Decision Ledger (Section 3.4).

2.5 How the benefits are demonstrated

Table 4. *Demonstrating the claimed benefits*

Benefit claimed	Where it shows in the demo
Agent autonomy	Segmentation chooses features, cluster count and which contacts stay as noise from an arbitrary CSV; the composer chooses which real offers to cite.
Responsible handling	The age-proxy flag on "Loyal Full-Price Buyers" appears on the Segments tab and as a REVISE in the Approval Queue.
Appropriate escalation	Planting a phone number, an unsubscribe-less footer or an unresolved merge tag in the builder turns the verdict to BLOCK and disables Send; the API refuses it with HTTP 409.
Traceability	The Logs tab separates "what I did" from "what the agents did"; each agent row shows its prompt and model version.
Scalability argument	Model calls per run depend on segment count, not contact count (Section 3.7).

3. System Architecture

3.1 Logical architecture

MOSAIC is organised in five layers (Figure 2). Dependencies point downward only: the console never talks to the database for pipeline state, and agents never talk to each other.

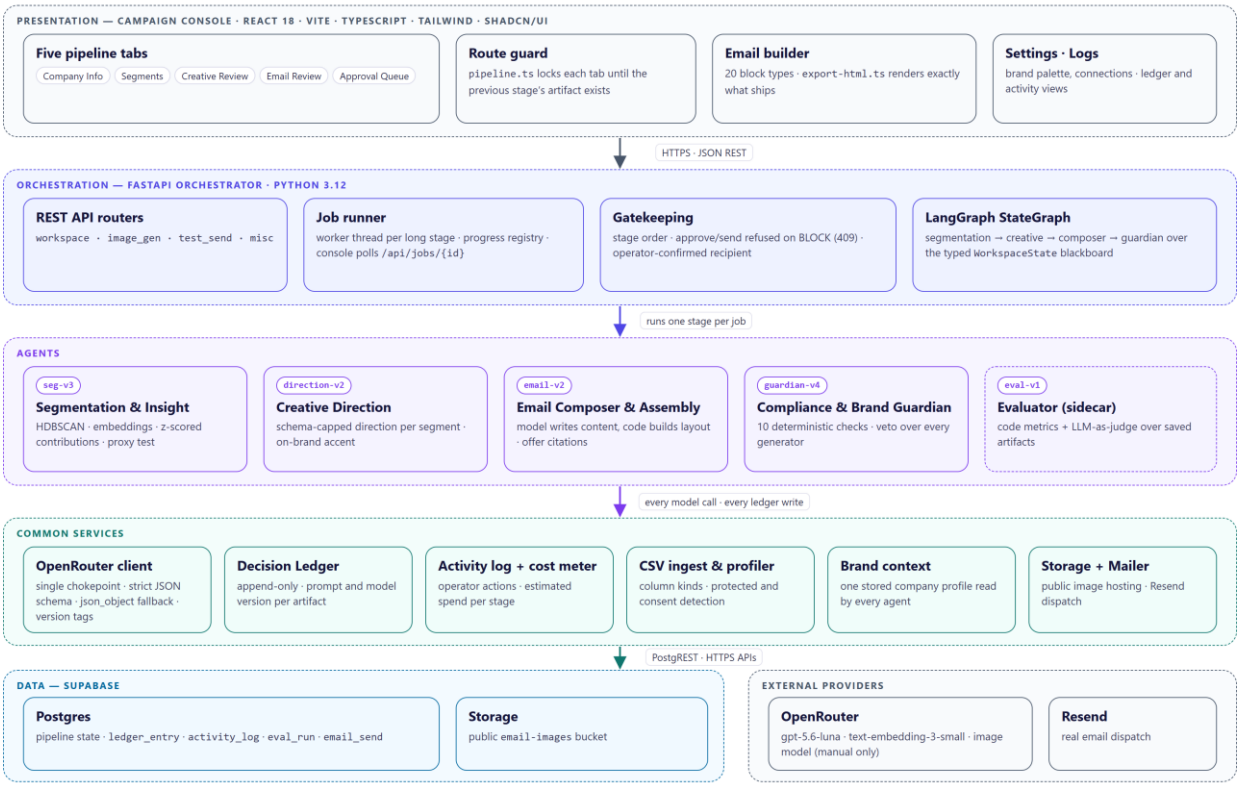


Figure 2. *Logical architecture.*

Table 5. *Logical layers*

Layer	Responsibility	Key components
Presentation	Operator workflow, review and editing	React 18 + Vite + TypeScript console; five pipeline tabs plus Settings and Logs; `pipeline.ts` route guard; drag-and-drop email builder with 20 block types and the `export-html.ts` renderer
Orchestration	Sequencing, job execution, gatekeeping	FastAPI app; routers `workspace`, `image_gen`, `test_send`, `misc`; job runner with progress registry; LangGraph `StateGraph` over `WorkspaceState`
Agents	The four decision-making agents plus the evaluator	`segmentation.py`, `creative_direction.py`, `email_composer.py`, `guardian.py`; `app/eval/` sidecar
Common services	Shared capabilities every agent uses	`openrouter_client.py` (single model chokepoint), `ledger.py`, `activity.py`, `cost_meter.py`, `csv_ingest.py`, `brand.py`, `storage.py`, `mailer.py`, `persistence.py`
Data and external	State and third-party capabilities	Supabase Postgres and Storage; OpenRouter; Resend

3.2 Physical architecture

Physically, MOSAIC is two deployable services built from one repository plus three managed services it does not host (Figure 3).

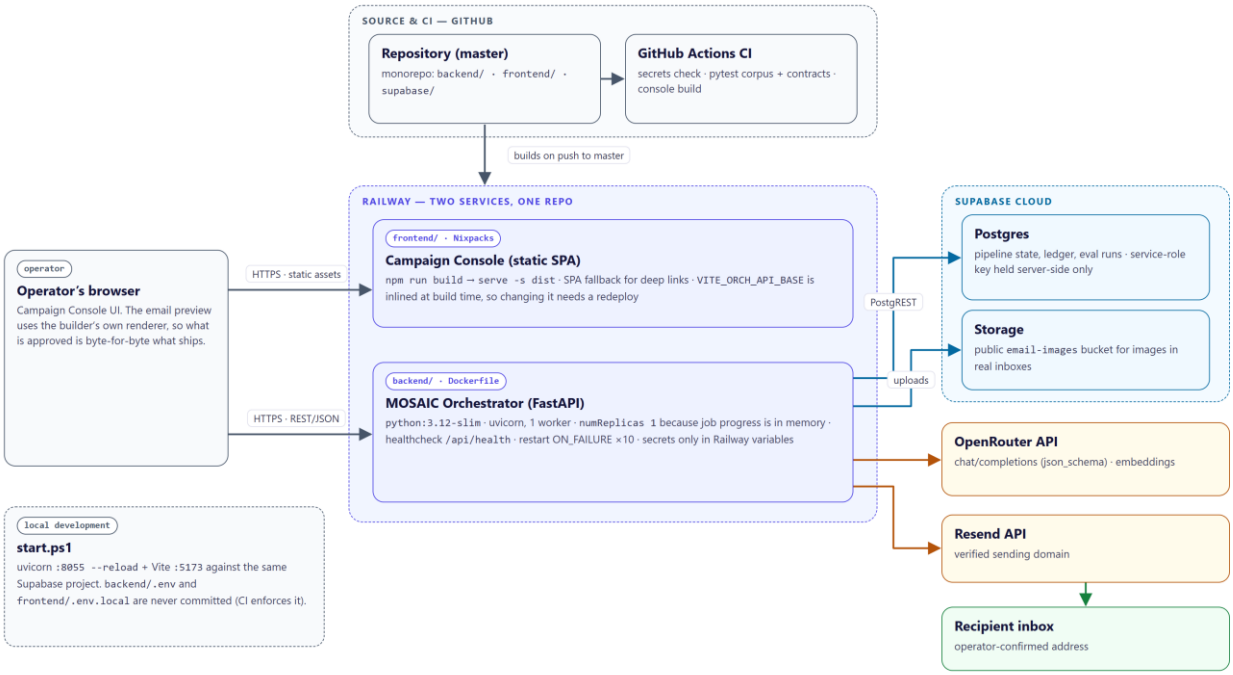


Figure 3. *Physical architecture and infrastructure.*

Table 6. *Infrastructure components*

Component	Technology	Hosting	Notes
Campaign Console	React 18, Vite, TypeScript, Tailwind, shadcn/ui	Railway service (Nixpacks), `serve -s dist`	Static SPA. `VITE_*` variables are inlined at build time, so changing the API base needs a redeploy.
MOSAIC Orchestrator	Python 3.12, FastAPI, uvicorn, LangGraph, scikit-learn, HDBSCAN	Railway service (Dockerfile, `python:3.12-slim`)	One worker, one replica; healthcheck `/api/health`; restart ON_FAILURE up to 10 times.
Database	Supabase Postgres	Supabase cloud	Schema in `supabase/migrations/0001–0008`; the backend writes with the service-role key.
Object storage	Supabase Storage, public `email-images` bucket	Supabase cloud	Hosts images so they render in real inboxes.
Models	OpenRouter (chat/completions, embeddings)	External API	Text, embeddings and judge model configured per role in `.env`.
Email dispatch	Resend	External API	Sends from a verified domain to an operator-confirmed address.
Local development	`start.ps1`	Developer machine	uvicorn on :8055 with reload, Vite on :5173, same Supabase project.

3.3 Deployment strategy

- **Containerisation.** The orchestrator ships as a Docker image based on `python:3.12-slim`. Every dependency has a manylinux wheel, so no compiler is needed; dependency installation is a separate layer so code-only changes rebuild quickly.
- **Two services, one repository.** Railway builds each service from its own root directory (`backend/` with its Dockerfile, `frontend/` with Nixpacks). The backend deploys first because the console needs its public URL at build time.
- **Health-gated releases.** Railway waits for `/api/health` to answer before routing traffic; a deployment that never becomes healthy is not promoted, and a crashed container restarts up to ten times.
- **Configuration and secrets.** All secrets (OpenRouter, Supabase service-role, Resend) live only in Railway variables or an untracked `backend/.env`. Nothing `VITE_`-prefixed may hold a secret, because it ships to the browser. CI fails if a `.env` file is ever tracked.
- **Single replica, deliberately.** Job progress is kept in process memory (`progress.py`), so a second replica would answer half of the console's polls with "job not found". `numReplicas: 1` encodes that; scaling out first needs a shared job store (Section 3.7).
- **Schema migrations.** Numbered SQL migrations are applied in order; the API turns a missing-table error into an actionable message naming the migration to run.

3.4 Agent communication protocols

MOSAIC uses a **blackboard** architecture rather than message passing. The Orchestrator threads a typed `WorkspaceState` through a LangGraph `StateGraph` (segmentation → creative direction → email composer → guardian). Each agent reads upstream artifacts and writes its own, and every artifact is persisted to Supabase, so each console tab can re-run its own stage from the latest persisted upstream output without re-running anything else (Figure 4).

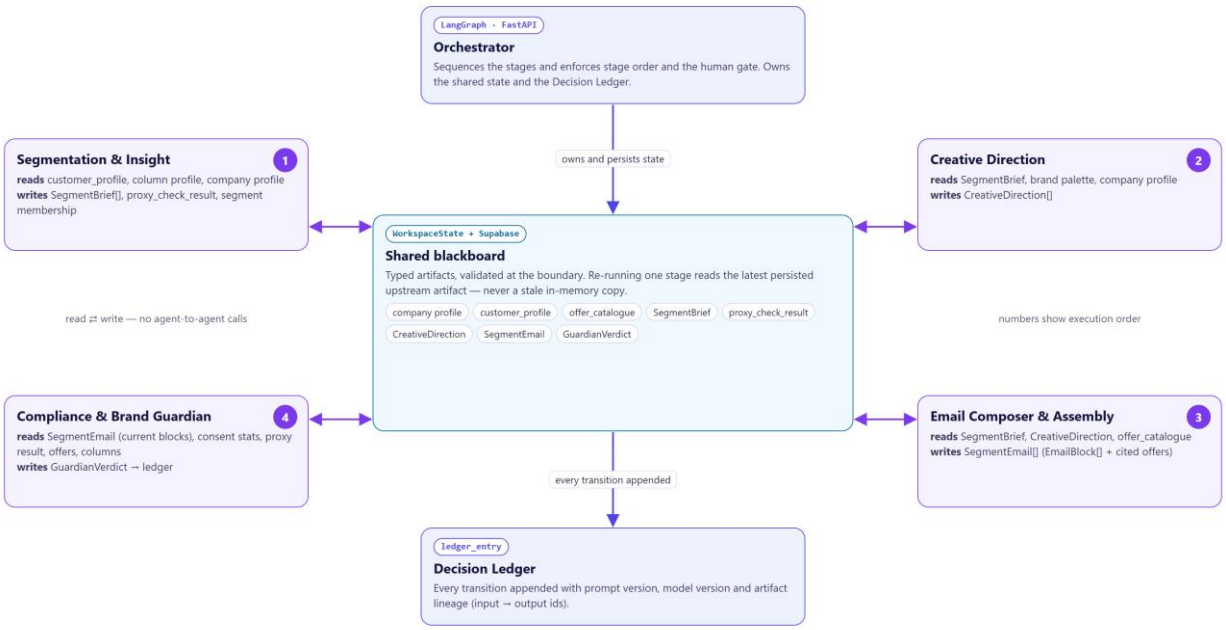


Figure 4. *Inter-agent communication through the shared blackboard.*

Table 7. *Typed artifacts on the blackboard*

Artifact	Producer	Consumers	Contract	Stored in
Company profile, column profile, offers	Operator + `csv_ingest.py`	All agents	Profile fields; column kinds and protected/consent flags	`app_setting`, `offer_catalogue`, `customer_profile`
`SegmentBrief`	Segmentation	Creative, Composer, Guardian, Evaluator	name, motivation, tone_guidance, size, confidence, feature_contributions	`segment_brief`
Proxy result	Segmentation	Guardian, console	protected_attribute, correlation, flagged	`proxy_check_result`
`CreativeDirection`	Creative Direction	Composer, Evaluator	tonality, language_style, cta_style, design_style, palette, logo/hero URLs, rationale	`creative_direction`
`SegmentEmail`	Composer (then operator edits)	Guardian, console, Evaluator	subject, preheader, `EmailBlock[]`, cited offer ids	`segment_email`
`GuardianVerdict`	Guardian	Approval Queue, send endpoint, Evaluator	verdict, per-check name/category/passed/detail/severity	`ledger_entry` (checks column)

Three further protocols complete the picture:

- **Model I/O protocol.** Every model call goes through `generate_text()`, which requests OpenAI-style `response_format: json_schema` in strict mode. Schemas are rewritten into the strict subset automatically; if a provider rejects strict mode, the call is retried once in `json_object` mode with the schema restated in the prompt. Outputs are therefore typed at the boundary, and a malformed or injected payload fails there instead of spreading.
- **Job protocol.** Long stages return a job id immediately; the work runs on a worker thread that reports named steps through an `on_step()` callback, and the console polls `GET /api/jobs/{id}` (Figure 5). No HTTP request is held open, so no gateway timeout can cut a run short.
- **Ledger protocol.** Every agent transition appends a `LedgerEntry` with stage, agent, prompt version, model version, input and output artifact ids and, for the Guardian, the verdict and full check results. Rows are never updated; the workspace reset is the only code path permitted to delete them.

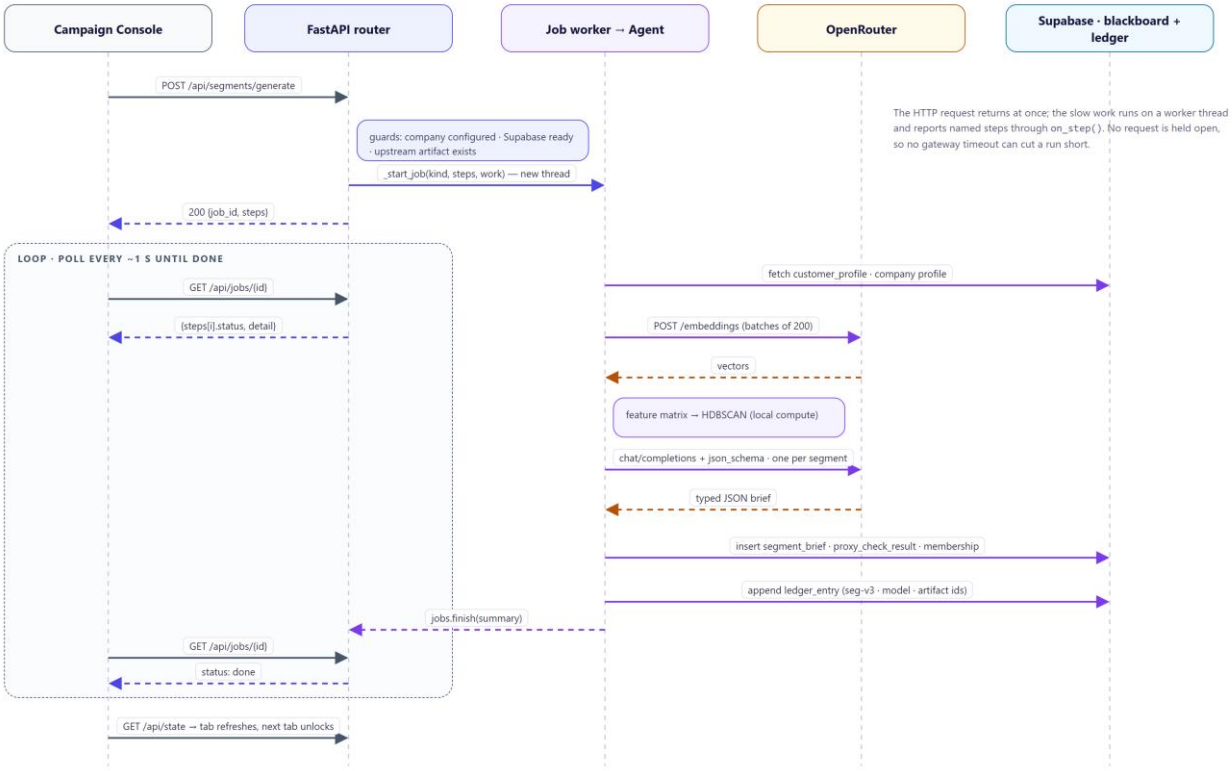


Figure 5. *Sequence of one stage run (segmentation), showing the asynchronous job protocol.*

3.5 Data architecture

Supabase Postgres is the single store for pipeline state, audit and evaluation (Figure 6). Customer attributes are stored as three JSONB groups — structured, behavioural and psychographic — so that any uploaded CSV fits without schema changes; free text and consent state get their own columns because they are treated specially.

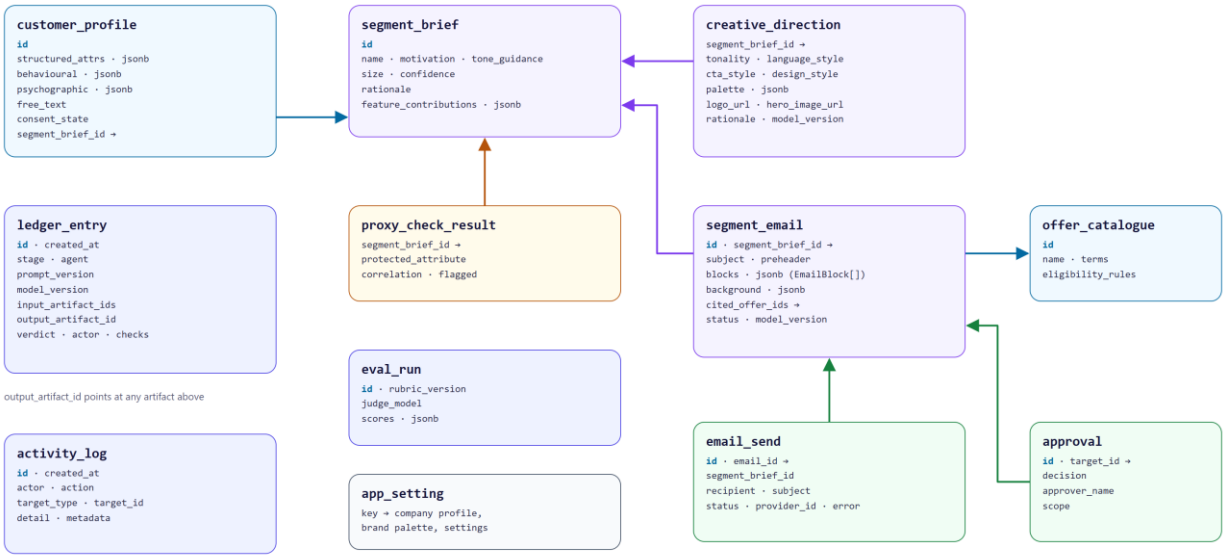


Figure 6. *Core data model (pipeline, audit and evaluation tables).*

3.6 Justification of architectural styles and technology choices

Table 8. *Architectural decisions*

Decision	Choice	Alternatives considered	Rationale
Coordination style	Blackboard with typed artifacts	Direct agent-to-agent messaging; one monolithic chain	Each stage is reviewable and re-runnable alone; no agent can influence another except through a validated artifact; lineage is natural to record.
Orchestration runtime	LangGraph `StateGraph` + FastAPI job runner	Hand-written pipeline; Celery	Explicit, inspectable graph over a typed state; stages also callable one at a time from the console. Celery would add a broker for a single-process demonstrator.
Graph shape	Strictly sequential	Parallel creative ‖ copy fan-out (proposal)	Creative direction became the composer's input, and each tab reviews the previous output before the next runs.
Clustering	HDBSCAN (leaf selection)	k-means; Gaussian mixtures	No need to fix k in advance; contacts may remain noise instead of being forced into a segment; membership probabilities give a confidence score.
Guardian implementation	Deterministic rule engine	LLM-as-judge in the veto path	Cheap, fast, testable in CI without network access, and explainable check by check. The LLM judge is used only for offline evaluation.
Model access	OpenRouter behind one client	Direct provider SDKs	One key, one chokepoint for schema enforcement, cost metering and version tagging; models swappable by configuration.
Layout generation	Code assembles typed `EmailBlock[]`	Model generates HTML/MJML	A bad model response can shorten copy but never produce an unrenderable or off-brand template; output drops straight into the builder.
Data store	Supabase Postgres + Storage	SQLite; separate object store	Managed Postgres with JSONB, row-level security and a public bucket for email images in one service.
Frontend	React + Vite + shadcn/ui	Streamlit	A drag-and-drop email builder and per-check review UI needed a real component framework.

3.7 Scalability

We justify scalability from the architecture rather than quoting a throughput figure we have not measured. Table 9 counts external model calls per full pipeline run directly from the code, for n contacts and k segments.

Table 9. *Model calls per run*

Stage	LLM (chat) calls	Embedding calls	Per-recipient work
Segmentation & Insight	k	⌈n / 200⌉ (only if the CSV has free text)	—
Creative Direction	k	—	—
Email Composer & Assembly	k	—	—
Guardian	0 (rule engine)	—	—

Stage	LLM (chat) calls	Embedding calls	Per-recipient work
Dispatch	0	—	Merge-tag fill from the contact record
Total	3k — 18 at the default cap of k = 6	5 at n = 1,000; 1,000 at n = 200,000	Deterministic

Chat-model work — the expensive, high-variance part — is therefore independent of list size: a 200,000-contact list costs the same 18 LLM calls as a 1,000-contact list at k = 6. Only the batched embedding calls grow with n. The demonstrator itself has been run end to end at 1,000 contacts; we have **not** load-tested it. Current limits are explicit:

- one backend replica, because job progress is held in memory;
- a 20,000-row ceiling per upload (`MAX_ROWS`) and a 25 MB request limit;
- HDBSCAN runs in-process on one machine; very large lists would need clustering on a sample with the remaining contacts assigned to the nearest cluster.

The path to horizontal scale is a shared job store (for example Redis or a Postgres jobs table), which removes the single-replica constraint, followed by moving long stages to a worker pool.

4. Agent Roles and Design

All four agents follow the same pattern: gather context from the blackboard, make one bounded decision, emit a schema-typed artifact, append a ledger entry. We deliberately chose **bounded autonomy**. Agents make real, data-driven decisions, but none of them runs an open-ended tool-calling loop, and the human operator is the outer loop: review, edit, regenerate, re-check. Section 9 discusses the bounded Guardian-to-Composer revise loop we would add next.

4.1 Orchestrator

Aspect	Design
Purpose	Sequence the stages, own the shared state and the Decision Ledger, and enforce stage order and the human gate.
Owner	OQQ
Implementation	`orchestrator/graph.py` (LangGraph `StateGraph` over `WorkspaceState`), `routers/workspace.py` (job runner, guards), `ledger.py`, `progress.py`.
Autonomy	The least autonomous component by design: it plans over a fixed action space and never chooses content.
Memory	`WorkspaceState` (typed in-run state) plus persisted artifacts in Supabase; the ledger is the audit memory.
Safeguards	Refuses a stage when its prerequisites are missing (no company profile, no Supabase, no upstream artifact); refuses approve or send for a BLOCK verdict (HTTP 409); an edit revokes any prior approval and triggers re-adjudication.

4.2 Segmentation & Insight Agent

Aspect	Design
Purpose	Discover micro-segments in the operator's own customer list and explain each one in business terms.
Owner	OQQ · prompt version `seg-v3`
Models	HDBSCAN (scikit-learn ecosystem) for clustering; `openai/text-embedding-3-small` for free text; `openai/gpt-5.6-luna` to name and describe each cluster.
Tools	Column profiler, feature-schema inference, StandardScaler, PCA, z-scored feature contributions, protected-attribute proxy test, Supabase profile queries.
Memory	Reads the stored column profile and company profile; writes `segment_brief`, `proxy_check_result` and each contact's segment membership.
Autonomy	Decides from the data which columns are features, how many clusters exist, and which contacts stay as noise. Keeps at most `MAX_SEGMENTS` (default 6) largest clusters.

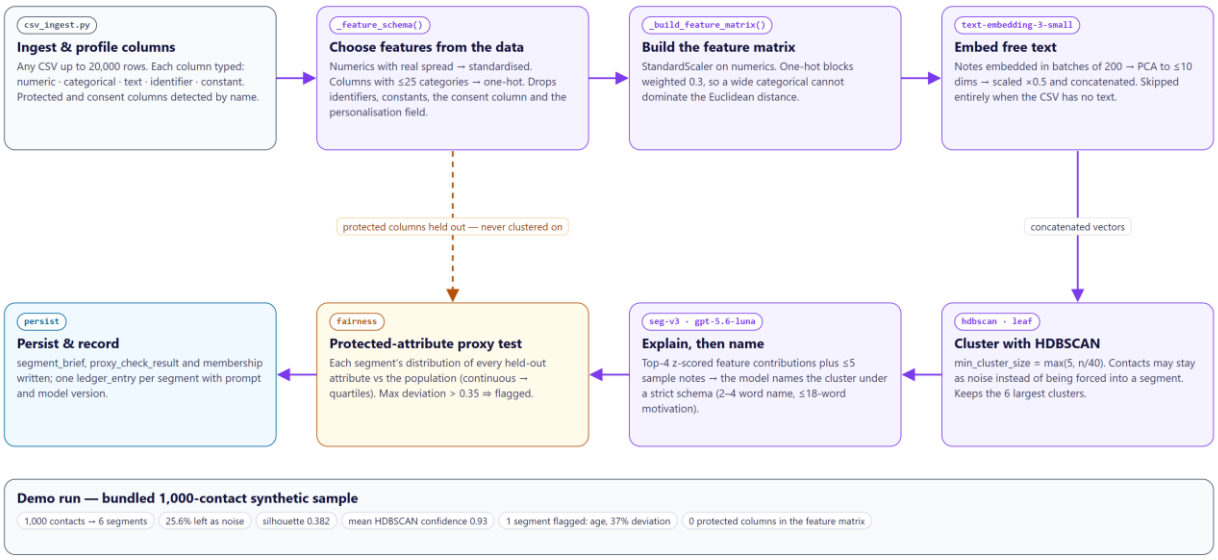


Figure 7. *Segmentation & Insight agent: internal pipeline.*

Reasoning pattern — explain, then name. The agent separates discovery from description. Discovery is numerical: numeric columns with real spread are standardised, categorical columns with at most 25 values are one-hot encoded at weight 0.3 (so a wide categorical block cannot dominate Euclidean distance), and text embeddings are reduced by PCA to at most 10 dimensions and scaled by 0.5. HDBSCAN runs with `min_cluster_size = max(5, n/40)` and leaf selection, which on the original base kept behavioural archetypes separate where the default merged them. Only then is the model asked to describe each cluster, and it is given the cluster's top-four z-scored feature contributions and up to five sample notes — not the raw rows.

Prompt pattern. Company context, then the distinguishing traits as JSON, then the sample notes, then a terse instruction. The schema caps the output to a 2–4 word name, one motivation sentence of at most 18 words, one tone phrase of at most 10 words and one rationale sentence of at most 22 words.

Fallbacks. No contacts or no clusterable columns raises an actionable error. If the CSV has no free text, embeddings are skipped rather than paying for empty strings. If HDBSCAN finds no dense cluster, the agent says so and suggests a larger or more varied list. If the naming call fails, the cluster is still saved, named "Segment n" with the error recorded, so a model outage never discards the clustering.

Demo results. On the 1,000-contact sample the agent produced six segments covering 744 contacts (25.6% left as noise), with mean membership confidence 0.93 and silhouette 0.382.

4.3 Creative Direction Agent

Aspect	Design
Purpose	Turn each segment into a short written creative direction that the composer must follow.
Owner	ABP · prompt version `direction-v2`
Model	`openai/gpt-5.6-luna` under a strict JSON schema.
Tools	Company context (`brand.py`), stored brand palette, hex validator, per-segment asset overrides.
Memory	Reads `SegmentBrief` and the brand palette; preserves operator overrides of logo and hero across regenerations (`existing_assets`); writes `creative_direction`.

Aspect	Design
Autonomy	Chooses tonality, language style, CTA style, design language and an accent colour for each segment.

Prompt pattern. A role statement (“creative director … no imagery is being produced”), the company context, the segment’s name, motivation and tone guidance, and the brand palette, followed by explicit constraints: each style field is a phrase of at most eight words; never suggest urgency, scarcity or loss framing; never reference age, gender or any protected attribute.

Fallbacks. The strict-schema subset cannot express a regex, so the accent colour is validated in code and falls back to the brand palette’s accent if it is not a valid hex value. If the call fails, the direction is saved with empty fields and the error in its rationale, visible on the Creative Review tab.

Change from the proposal. The proposal’s Creative Director generated images and critiqued them in a loop. We replaced that with a written direction because per-segment image generation was expensive, slow to review and hard to evaluate. Image generation still exists, but only as an operator-triggered tool.

4.4 Email Composer & Assembly Agent

Aspect	Design
Purpose	Write each segment’s email and assemble it into the builder’s block format.
Owner	OQQ · prompt version `email-v2`
Model	`openai/gpt-5.6-luna` under a word-capped strict JSON schema.
Tools	Offer catalogue, block assembler (`_build_blocks`), merge-tag layer (`brand.merge_tag`), company social and footer links.
Memory	Reads `SegmentBrief`, `CreativeDirection` and the offer catalogue; writes `segment_email` with its blocks, background and cited offer ids.
Autonomy	Chooses the copy and which real catalogue offers to cite; code owns the layout.

Split of responsibility. The model writes only content: subject (≤8 words), preheader (≤12), headline (≤7), intro (≤34), exactly three highlight cards, a 2–3 word CTA, two FAQs and a closing line. Code assembles the layout — centred logo, hero, headline, intro, three benefit columns, an offer coupon, FAQ accordion, CTA, closing, social links and footer — and applies the segment’s palette and corner radius derived from its design language. A bad model response can shorten the copy but cannot produce an unrenderable or off-brand template.

Grounding enforced, not trusted. Cited offer ids that are not real catalogue rows are dropped before saving. The coupon block appears only when an offer is genuinely cited. The personalised greeting is inserted by code as a real merge tag, because a model asked to include `{{first_name}}` drops it often enough to matter.

Prompt pattern. Company context, segment, the creative direction to follow, the list of citable offers with ids and terms, then rules: no filler openers, no exclamation marks or emoji, never invent prices or offers outside the list, avoid urgency, scarcity and loss framing, never mention protected attributes, and never include an email address or phone number.

Fallbacks. A segment without a direction is skipped rather than composed off-brand. If the call fails, a clearly marked placeholder is saved; because it cites no offer, it cannot pass the Guardian's grounding check.

4.5 Compliance & Brand Guardian Agent

Aspect	Design
Purpose	Adjudicate every email before any send, with a veto over every generator.
Owner	ABP · check-set version `guardian-v4`
Model	None. A deterministic rule engine, by design.
Tools	Consent statistics per segment, proxy-test results, offer catalogue, uploaded column names, PII scanner, merge-tag resolver, link and image checkers.
Memory	Reads the email's current blocks (after any edit); writes its verdict and per-check results to the ledger. The console ignores verdicts from older check-set versions.
Autonomy	Issues PASS, REVISE or BLOCK; nothing in the system can override a BLOCK.

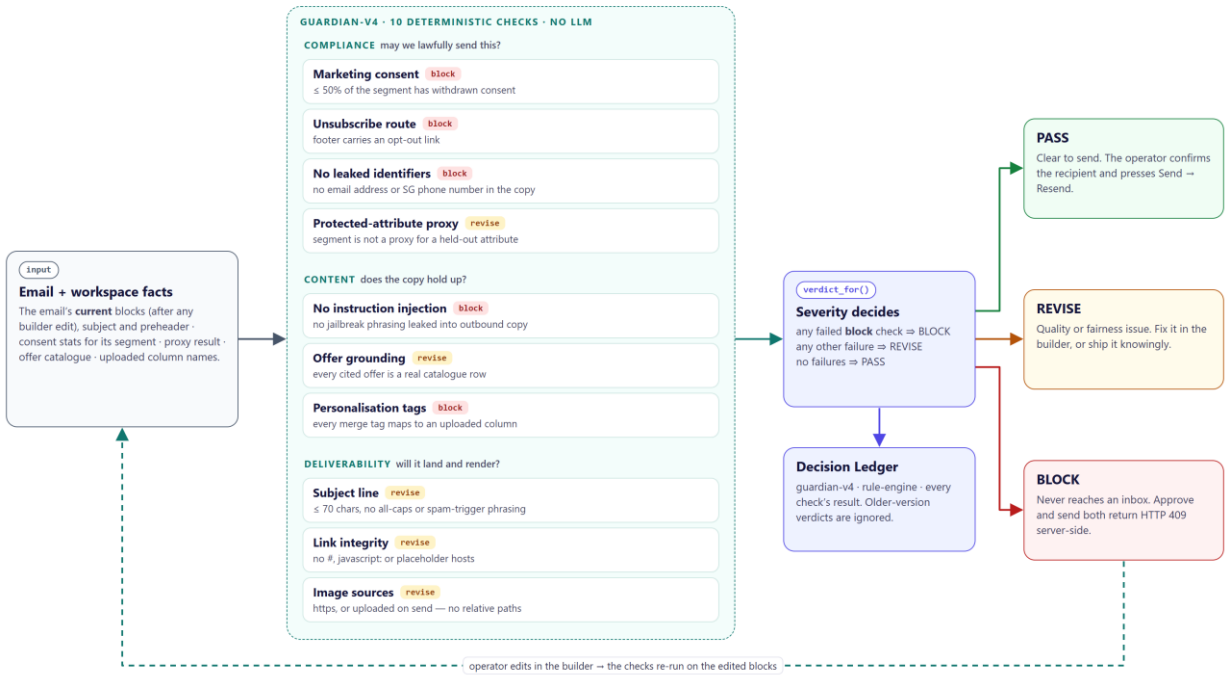


Figure 8. *Guardian: ten checks, severity-driven verdict and the re-check loop.*

Table 10. *Guardian check set (guardian-v4)*

Check	Group	Severity	Fails when
Marketing consent	Compliance	block	More than 50% of the segment's contacts have withdrawn consent
Unsubscribe route	Compliance	block	No footer block, or the footer carries no unsubscribe / opt-out / preferences link
No leaked identifiers	Compliance	block	An email address or Singapore phone number appears in the visible copy

Check	Group	Severity	Fails when
Protected-attribute proxy	Compliance	revise	The segment was flagged by the proxy test (max deviation > 0.35)
No instruction injection	Content	block	Jailbreak phrasing such as “ignore previous instructions” appears in the copy
Offer grounding	Content	revise	No offer is cited, or a cited id is not in the catalogue
Personalisation tags	Content	block	A `{{tag}}` or `%%tag%%` names no uploaded column, so it would render literally
Subject line	Deliverability	revise	Empty, longer than 70 characters, all caps, excessive punctuation or spam-trigger phrasing
Link integrity	Deliverability	revise	A link is `#`, empty, `javascript:`, a placeholder host or not http(s)/mailto/tel
Image sources	Deliverability	revise	An image uses a relative path or plain http

Verdict logic. Any failed block-severity check gives BLOCK; any other failure gives REVISE; no failures gives PASS. BLOCK means the message must never reach an inbox: sending without consent or an opt-out route is unlawful, leaking another customer’s identifiers is a data incident, and an unresolved merge tag renders literally to a real person. REVISE is a quality or fairness problem the operator may fix or knowingly ship over.

Why no LLM. Keeping a language model out of the veto path makes the Guardian fast, free to run on every edit, testable in CI without network access, and explainable check by check. Language-model judgement is used only in the offline evaluator.

4.6 Evaluator (sidecar)

The evaluator (`app/eval/`, rubric `eval-v1`) scores all four decision agents against artifacts that are **already saved**, never against a fresh and friendlier run. Each agent’s rubric sums to 100 and mixes deterministic code metrics with LLM-as-judge criteria scored on a 1–5 Likert scale by a judge whose system prompt instructs it to score conservatively and never invent facts. Runs are stored in `eval_run` with the judge model and a delta against the previous run. If the judge cannot run, the affected criteria are reported as unavailable rather than estimated. Results are in Section 8.

4.7 Shared prompt-engineering patterns and fallbacks

Table 11. *Shared patterns*

Pattern	Where	Purpose
Strict JSON schema with per-field word caps	All generating agents	Typed output at the boundary; injected instructions have nowhere to land; compact UI cards
Schema strictification + `json_object` **retry**	`openrouter_client.py`	Works across providers that do not support strict mode
Context first, constraints last	All prompts	Company context and segment facts, then explicit prohibitions (urgency, protected attributes, contact details)

Pattern	Where	Purpose
Model names, code decides	Segmentation, Composer	The model describes numbers it was given and writes copy; structure and grounding are enforced in code
Fail visible, not silent	All agents	Failures produce a marked artifact or an actionable error and are logged, never a plausible-looking fake
Version tags on every call	Ledger	Prompt and model version recorded for every artifact

5. Explainable and Responsible AI Practices

5.1 Alignment across the development and deployment lifecycle

Table 12. *Responsible-AI alignment by lifecycle stage*

Lifecycle stage	Principles	What is implemented	Evidence
Data collection and ingestion	Privacy, fairness, consent	Synthetic data only; consent column detected and stored as `consent_state`; protected columns flagged by whole-token name matching (so `average_basket` is not mistaken for "age")	`csv_ingest.py`; contract test `test_protected_columns_are_recognised`
Feature engineering	Fairness	Protected, consent and identifier columns excluded from the feature matrix; categorical weighting prevents one column dominating	`_feature_schema()`
Modelling (clustering)	Fairness, explainability	Z-scored feature contributions per segment; proxy test against every held-out attribute	Segments tab; `proxy_check_result`
Generation	Safety, non-manipulation, grounding	Schema caps; prohibitions in prompts; uncited offers dropped; layout owned by code	Agent prompts and schemas; contract tests
Adjudication	Accountability, compliance	Deterministic Guardian with BLOCK severity for legal and safety failures	`guardian.py`; LLMSecOps corpus in CI
Human review and dispatch	Human oversight	Only a person can send, to a confirmed address; BLOCK refused server-side; edits re-run the checks	Approval Queue; `/api/emails/{id}/send`
Monitoring and evaluation	Transparency, accountability	Append-only ledger with versions; activity log; eval-v1 sidecar including a "no dark patterns" criterion	Logs tab; `eval_run`

5.2 Fairness and bias mitigation

Exclusion. Protected attributes — age, sex, gender, race, ethnicity, religion, nationality, disability, pregnancy, marital status, date of birth and age bands — never enter the feature matrix.

Exclusion is not enough, so we test for proxies. Behavioural features can still encode a protected attribute. After clustering, the agent compares each segment's distribution of every held-out attribute with the population's. Continuous attributes are first bucketed into population quartiles, because a raw age column has a distinct value for almost every contact and would never flag. The largest absolute deviation is recorded; above 0.35 the segment is flagged. A flagged segment is shown on the Segments tab and carried into the Guardian as a REVISE on every email from that segment, so a person must consciously decide.

Table 13. *Proxy-test results on the demo workspace (protected columns present: age, age_bracket, gender)*

Segment	Contacts	Confidence	Worst attribute	Max deviation	Flagged
Lapsed Complaint Customers	141	0.913	age	0.192	No
Loyal Full-Price Buyers	134	0.934	age	0.372	Yes → REVISE
App-First Drop Explorers	134	0.930	age	0.277	No
Seasonal Gift Buyers	121	0.929	age	0.220	No

Segment	Contacts	Confidence	Worst attribute	Max deviation	Flagged
Conscious Cat Keepers	114	0.927	age_bracket	0.147	No
Discount-Driven Cat Shoppers	100	0.946	age_bracket	0.067	No

The finding is plausible: a segment defined by high spend and loyalty skews older. MOSAIC does not silently suppress the segment — that would be a judgement the system is not entitled to make — but it makes the correlation visible and requires a person to own the decision.

Bias in language. Names and descriptions come from the segment's own feature contributions, not from free association; prompts forbid references to protected attributes; and the evaluator scores segment-name faithfulness against the contributions.

No individual price discrimination. Offer value comes only from catalogue rows with defined eligibility. The composer can choose which real offer to cite but cannot invent a price, and nothing varies offer value by inferred willingness to pay.

5.3 Explainability

Table 14. *Explainability artifacts*

Artifact	Explanation provided	Where the operator sees it
Segment	Size, HDBSCAN confidence, top-four z-scored feature contributions (signed bars), one-sentence rationale, proxy result	Segments tab
Creative direction	Rationale linking the direction to the segment	Creative Review tab
Email	Cited catalogue offers; Guardian verdict broken down by category and check with a plain-language detail line	Email Review, Approval Queue
System behaviour	Ledger rows with stage, agent, prompt version, model version and artifact lineage; operator actions separated from agent actions	Logs tab ("What I did" / "What the agents did")
Agent quality	Per-criterion rubric breakdown, code vs LLM-judge, with delta from the previous run	Approval Queue → Model scores

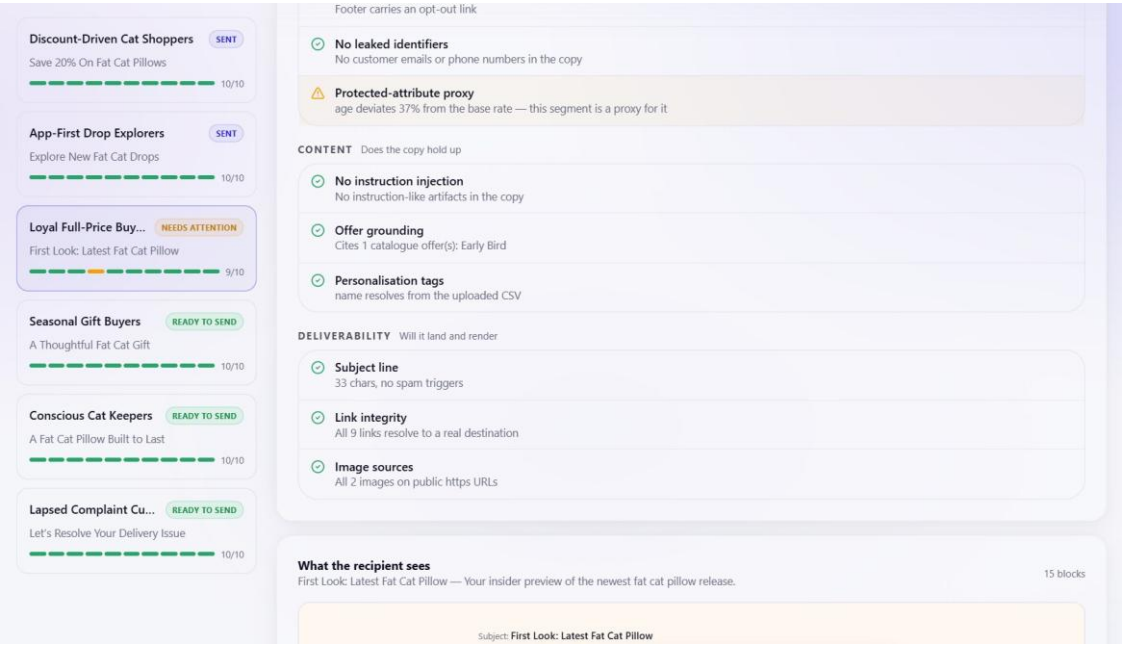


Figure 9. *Approval Queue: per-check Guardian breakdown for the REVISE email, with the proxy finding explained in plain language.*

5.4 Accountability and human oversight

- Human-in-the-loop at every stage: each artifact is reviewed before the next tab unlocks, and every artifact can be edited or regenerated.
- Only a person can send. There is no configuration flag that lets an agent dispatch, and nothing in the LangGraph graph sends mail.
- The gate is enforced server-side: approve and send both return HTTP 409 for an email whose latest verdict is BLOCK, independently of the disabled button in the UI.
- Editing an email revokes any earlier approval and re-runs the Guardian, so an approval always refers to the artifact that actually ships.
- Limitation: approvals are recorded against a generic “Operator” identity because the demonstrator has no authentication (Progress Report 1 discussed this trade-off). Named approvers are the first step of the auth work in Section 9.

5.5 Manipulation guardrail and ethical exclusions

The proposal planned a separate manipulation-index scorer. In the build, the guardrail is layered instead: generation prompts forbid urgency, scarcity and loss framing; the subject-line check flags spam-trigger phrasing and shouting; and the evaluator’s Creative rubric scores “no dark patterns” (the demo direction set scored 20 out of 20). Four capabilities are excluded on ethical grounds:

- no targeting or creative variation on inferred health status, financial distress, bereavement or any protected attribute;
- no individual price discrimination — offer value varies only by catalogue eligibility rules;
- no targeting of emotional vulnerability, including send timing;
- no generated imagery of identifiable real people, and no fabricated endorsements or testimonials.

5.6 Governance framework alignment

We mapped MOSAIC to the four areas of Singapore's Model AI Governance Framework (IMDA/PDPC, second edition) and to the consent obligations of the PDPA.

Table 15. *Governance mapping*

Framework area	How MOSAIC addresses it
Internal governance structures and measures	Named ownership of each agent (OQQ / ABP), with the Guardian owned by the member who did not build the generators; a mandatory human gate; versioned check sets and rubrics; CI as a policy gate.
Level of human involvement in AI-augmented decision-making	Risk-tiered human-in-the-loop: the system drafts and checks, a person decides; BLOCK cannot be overridden; REVISE requires a conscious choice.
Operations management	Data provenance (synthetic, profiled columns); protected-attribute handling and proxy testing; prompt and model versions on every ledger row; evaluation runs with deltas; deterministic tests in CI.
Stakeholder interaction and communication	A working unsubscribe route is a BLOCK-level requirement; every email carries an explanation trail; the console explains each check in plain language.
PDPA — consent and withdrawal	Consent column detected at upload; withdrawal is honoured whenever the Guardian runs, including on every edit, so a withdrawal mid-campaign is picked up before the next send.

5.7 Data minimisation

The composer receives no customer records at all — only the segment description, the direction and the offer list — and merge tags are filled locally at send time. The segmentation naming call receives aggregate z-scores and at most five sample notes. Free-text notes are sent to the embeddings endpoint; with real customer data this would require a data-processing agreement with the provider or PII scrubbing before embedding (risk R4 in Section 6).

6. AI Security Risk Register

6.1 Threat model

MOSAIC ingests text an attacker can influence — free-text profile fields in an uploaded CSV — and then emits content to third parties under a brand's name. The trust boundaries are: the uploaded CSV (untrusted), model output (untrusted until checked), the operator (trusted, but able to paste untrusted content into company fields), and three external services. Assets at risk are the brand's reputation, recipients' personal data and consent rights, the operator's API budget, and the integrity of the audit trail.

Likelihood (L) and impact (I) are each rated Low (L), Medium (M) or High (H). Risks are mapped to the OWASP Top 10 for LLM Applications (2025) where one applies; the residual-risk column states what remains after the implemented controls and what we would do next.

Table 16. *AI security risk register*

ID	Risk	OWASP	L / I	Controls implemented	Evidence	Residual risk and planned action
R1	Indirect prompt injection via customer free text	LLM01	M / H	Free text reaches the chat model only as ≤5 sample notes in a data position; strict, length-capped JSON schema leaves no free-form channel; Guardian BLOCKs instruction-like phrasing in output	`test_prompt_injection_in_copy_blocks`	Medium. No input sanitiser or delimiting yet; add structural delimiting and an ingest-time injection scan.
R2	Injection through operator-entered fields (company description, intent, offer terms)	LLM01	L / M	Same schema caps and output scan; the operator is a trusted user in the demo	Guardian corpus	Low.
R3	Cross-recipient PII leakage in copy	LLM02	L / H	Composer receives no customer records; prompt forbids contact details; Guardian email/phone scan is BLOCK	`test_pii_leak_blocks`; CI negative test	Low.
R4	Personal data sent to the model provider	LLM02	M / M	Data minimisation (Section 5.7); synthetic data only	Prompt construction in code	Medium for real data: scrub PII before embedding or use a provider under a DPA.
R5	Hallucinated offers, prices or claims	LLM09	M / M	Uncited ids dropped; grounding check; coupon only when cited; judge scores grounding	`test_ungrounded_offer_fails_grounding`	Low–Medium. Grounding is REVISE, not BLOCK; consider raising severity.
R6	Improper output handling (model text rendered as HTML)	LLM05	L / M	Model output placed into typed block fields and stringified; builder sanitises HTML with DOMPurify and validates URL schemes; link check rejects `javascript:`	`email-sanitize.ts`; link check	Low.
R7	Excessive agency	LLM06	L / H	No agent can send; sending is a human action to a confirmed address; BLOCK refused server-side	Send endpoint; graph has no dispatch node	Low. Send currently accepts an email with no current-version verdict; require one.
R8	Unbounded consumption / cost exhaustion	LLM10	M / M	`MAX_SEGMENTS = 6`; no generation loops; 3k chat calls per run; cost meter per stage	Cost meter; Section 3.7	Medium. No hard budget ceiling or rate limit; add both.

ID	Risk	OWASP	L / I	Controls implemented	Evidence	Residual risk and planned action
R9	Unresolved merge tag or wrong personalisation	—	M / M	Both tag dialects resolved against uploaded columns; unresolved tag is BLOCK; tags filled from a real segment contact at send	`test_unresolved_merge_tag_blocks`	Low.
R10	Sending to contacts who withdrew consent	—	L / H	Consent check is BLOCK above 50% withdrawn; re-checked on every Guardian run	Consent check	Low in the demo (single confirmed recipient). Bulk sending must filter withdrawn contacts per recipient.
R11	Segments acting as proxies for protected attributes	—	M / M	Protected columns held out; proxy test; REVISE in Guardian	Table 13	Medium. A person can ship a flagged segment knowingly; record the reason.
R12	Secret leakage and dependency supply chain	LLM03	M / H	Secrets only server-side; CI rejects tracked `.env` files and live key assignments	CI job 1; `test_no_dotenv_or_live_keys_in_repo`	Medium. Dependencies use `>=` ranges; pin with hashes and add `pip-audit` / `npm audit` to CI.
R13	Unauthenticated API and no rate limiting	—	M / H	CORS allow-list; secrets not exposed to the browser	`config.py`	High. Add authentication with named approvers and per-route rate limits before any non-demo exposure.
R14	System prompt leakage	LLM07	L / L	Prompts contain no secrets or credentials	Prompt source	Low.

6.2 AI security testing

The LLMSecOps corpus in `backend/tests/fixtures/llmsecops/` holds six fixture emails run through the real `run_checks()` and `verdict_for()` path with external dependencies stubbed: a clean email (must PASS), a PII leak, instruction-injection phrasing, a missing unsubscribe route and an unresolved merge tag (each must BLOCK), and an ungrounded offer (must fail grounding). A deliberate negative test on branch `ci/demo-fail-pii` planted a phone number and email address in the clean fixture; the CI job failed with `assert 'BLOCK' == 'PASS'`, which shows the gate refuses to treat a PII leak as clean.

What we have not done is adversarial red-teaming of the live models — for example, crafting CSV notes designed to steer segment names. That is the next security test to add, together with the input sanitiser in R1.

7. MLSecOps / LLMSecOps Pipeline

7.1 Pipeline overview

Every push and pull request runs `.github/workflows/ci.yml`. No CI job calls OpenRouter or Supabase, so the gate is deterministic, free and safe to run on every change (Figure 10).

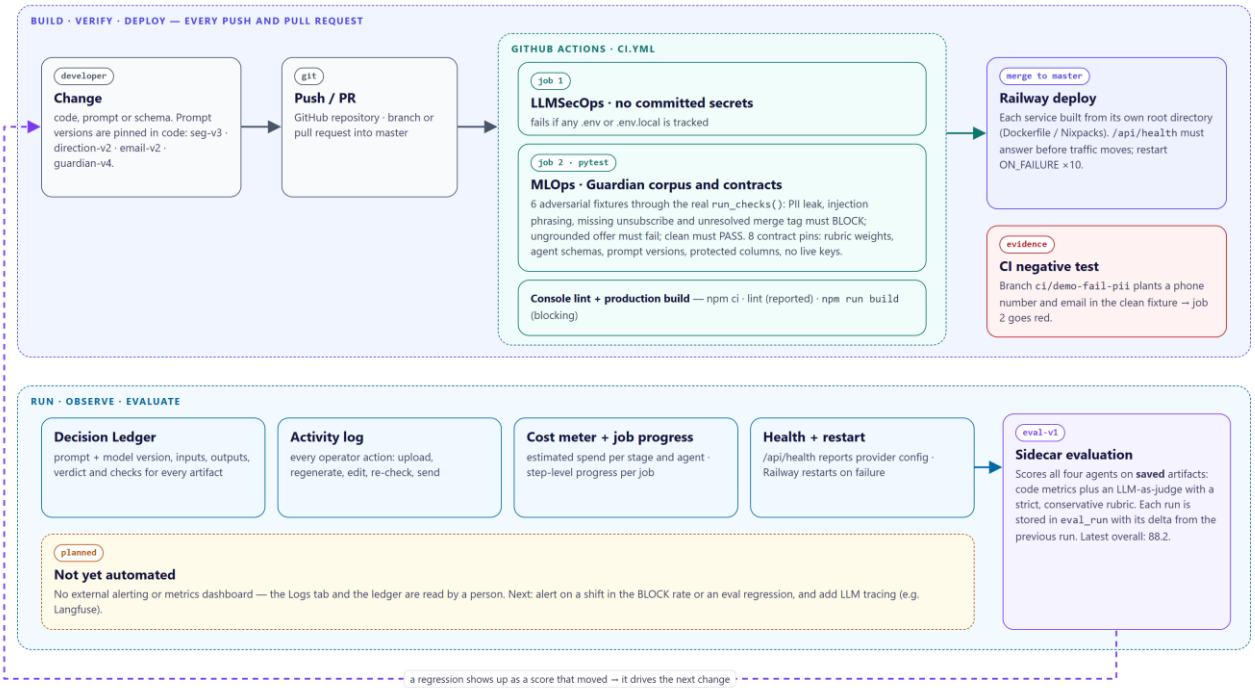


Figure 10. *MLSecOps / LLMSecOps pipeline: build, verify, deploy, then run, observe and evaluate.*

7.2 Automated testing, including AI security tests

Table 17. *CI jobs*

CI job	What it locks	Fails when
Job 1 — LLMSecOps · no committed secrets	Secret hygiene	Any `.env` or `.env.local` file is tracked
Job 2 — MLOps · Guardian corpus and contracts	Guardian behaviour on six adversarial fixtures; eval-v1 weights sum to 100 per agent; agent schemas still require their artifact fields; prompt versions `seg-v3`, `direction-v2`, `email-v2`, `guardian-v4` pinned; protected column names recognised; no live API key assignments in tracked code	Any of the 14 pytest assertions fails
Job 3 — Console lint and production build	The exact build command Railway runs	`npm run build` fails (lint is reported but not blocking, because of pre-existing builder warnings)

7.3 Versioning and tracking

Table 18. *Versioning*

Artefact	How it is versioned	Where it is recorded
Code	Git; pull requests gated by CI	GitHub
Prompts	Version string per agent in code (`seg-v3`, `direction-v2`, `email-v2`); asserted by a contract test so a silent change fails CI	Every ledger row
Models	Model id per role in configuration	Every ledger row; creative and email artifacts also store `model_version`
Guardian check set	`GUARDIAN_VERSION = guardian-v4`; the console ignores verdicts from older versions	Ledger (`prompt_version` of Guardian rows)
Evaluation rubric	`RUBRIC_VERSION = eval-v1`; weights pinned by test	`eval_run.rubric_version`, plus the judge model
Database schema	Numbered migrations `0001`–`0008`	`supabase/migrations/`

7.4 Deployment strategy

Merges to `master` are built by Railway, one service per root directory. The orchestrator image must pass its `/api/health` check before it receives traffic, and crashed containers restart automatically. Configuration is environment-based, with secrets held only in Railway variables. Rollback is by redeploying a previous Railway deployment. Section 3.3 gives the details.

7.5 Monitoring and alerting

Table 19. *Monitoring signals*

Signal	Source	Status
Service health and provider configuration	`GET /api/health`; Railway healthcheck	Implemented
Per-stage model spend	In-process cost meter, shown on the Logs tab	Implemented (estimates; resets on restart)
Guardian verdict distribution and per-check pass rates	Ledger; eval-v1 Guardian metrics	Implemented, read by a person
Agent quality regressions	`eval_run` with delta against the previous run	Implemented, run on demand
Alerts on BLOCK-rate shift, eval regression, provider error rate, stage latency	—	Planned
LLM tracing (prompts, latencies, token usage)	—	Planned (e.g. Langfuse)

A shift in the BLOCK rate is the earliest signal that something upstream has drifted — a new model version, a prompt change or a different kind of customer list — so it is the first alert we would automate.

7.6 Logging and auditability

- **Decision Ledger** (`ledger_entry`): append-only, one row per agent transition, with prompt and model versions, input and output artifact ids, verdict and check results. If the database is unavailable, entries fall back to an in-process list and a log line rather than being lost silently.
- **Activity log** (`activity_log`): every operator action — upload, regenerate, edit, re-check, send, evaluation run — with actor, target and detail.
- **Dispatch records** (`email_send`): every send attempt, successful or refused by the provider, with recipient, subject, status and provider id or error.
- **Application logs**: Python logging of job failures with stack traces, visible in Railway's log stream.

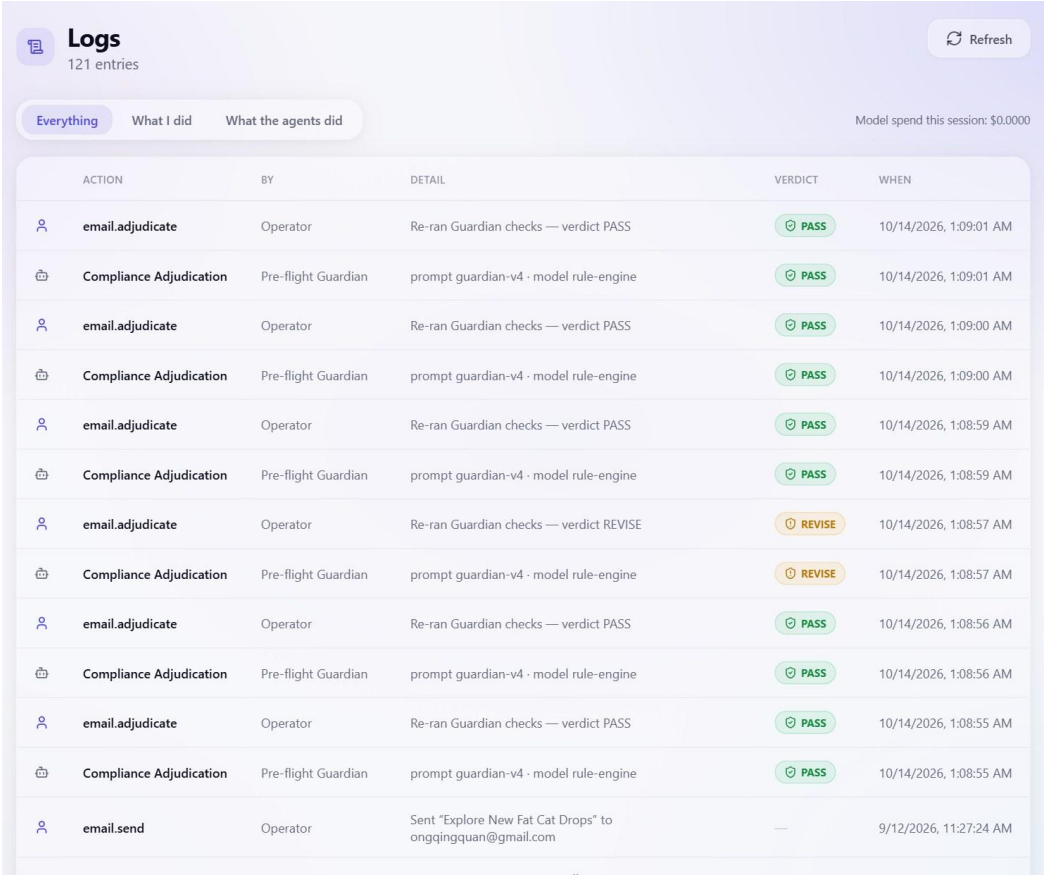


Figure 11. *Logs tab: agent ledger rows (prompt and model version, verdict) interleaved with operator actions.*

8. Testing and Evaluation Summary

8.1 Types of tests performed

Table 20. *Tests and evaluations performed*

Type	Scope	How it runs	Result
Contract tests (MLOps)	Rubric weights, agent output schemas, prompt-version pins, protected-column detection, secret scan	pytest, local and CI	8 / 8 pass
AI security tests (LLMSecOps)	Guardian on six adversarial fixtures	pytest, local and CI	6 / 6 pass
CI negative test	Planted PII in the clean fixture	Branch `ci/demo-fail-pii`	Build failed, as intended
End-to-end integration run	1,000-contact CSV → 6 segments → 6 directions → 6 emails → Guardian → dispatch	Console against live Supabase, OpenRouter and Resend (12 Sep 2026)	Completed; 3 live sends delivered
Guardian re-adjudication	All six emails re-checked with guardian-v4	API (14 Oct 2026)	5 PASS, 1 REVISE, 0 BLOCK
Agent quality evaluation	All four decision agents on saved artifacts	eval-v1 sidecar, judge `openai/gpt-5.6-luna`	88.2 overall

8.2 Results



Figure 12. *eval-v1 scores by agent (12 September 2026 run on the demo workspace).*

Table 21. *eval-v1 breakdown*

Agent	Score	Criterion breakdown (earned / max)
Segmentation	70.8	Silhouette 26.6/38.5 (0.382) · Coverage 13.0/17.5 (25.6% noise) · HDBSCAN confidence 13.0/14 (0.93) · Faithfulness 6.9/15 (judge) · Specificity 11.2/15 (judge)
Creative Direction	95.0	Four length checks 8/8 each · Accent hex 8/8 · Segment fit 20/20 · Brand fit 15/20 · No dark patterns 20/20
Email Composer	93.3	Has citation 10/10 · Citations in catalogue 10/10 · Subject length 10/10 · Preheader length 10/10 · Follows direction 20/20 · Segment fit 19.2/20 · Grounded 14.2/20

Agent	Score	Criterion breakdown (earned / max)
Guardian	93.7	Check pass rate 68.7/70 (all checks 100% except the proxy check at 83.3%) · Email PASS rate 25/30 (5 of 6)
Overall	88.2	Mean of the four agent scores

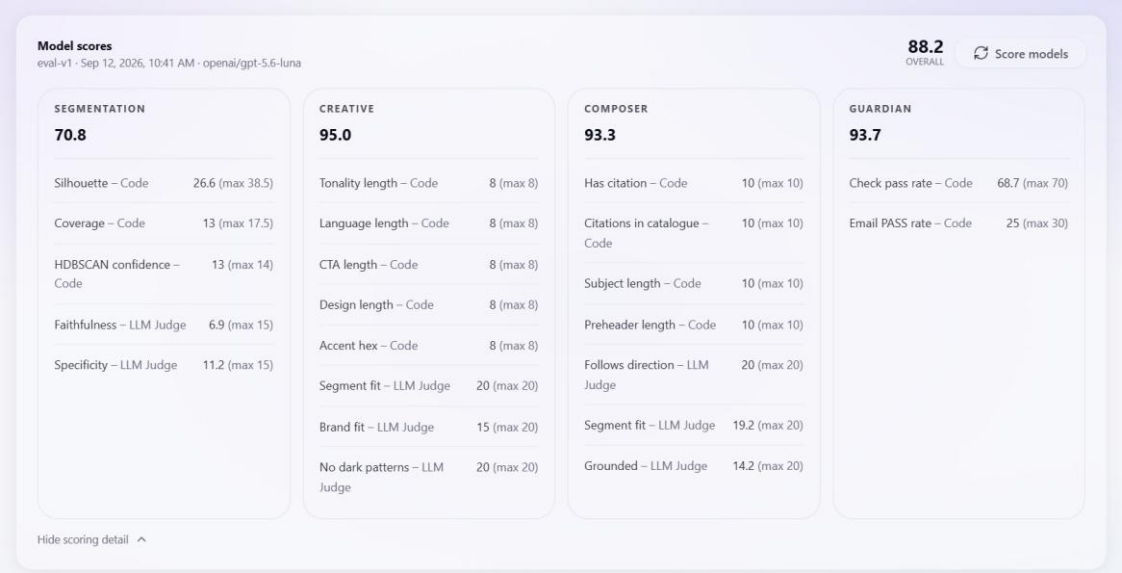


Figure 13. *The same scores as shown in the console's Model scores panel.*

8.3 Key findings

- **Segmentation is the weakest agent and the most honest score.** A silhouette of 0.382 and 25.6% noise reflect overlapping synthetic archetypes — Progress Report 1 records that we deliberately added noise and overlap so clustering had something real to fail at. The lowest single criterion is segment-name faithfulness (6.9 / 15): the judge found that some names claim more than the contributions show. This is our clearest improvement target.
- **Generation is well constrained.** Every deterministic check on the creative and composer output passed, which is what the schemas and code-owned layout were designed to achieve. The judge's lowest generation score is grounding (14.2 / 20), consistent with keeping grounding failures visible rather than trusting the model.
- **The Guardian behaves as designed.** It caught the one fairness issue present (age proxy) and nothing spurious.
- **Limitations of the evaluation.** The LLM judge has not yet been calibrated against a human-labelled sample, as the proposal intended; results come from a single evaluation run on one workspace; the generating agents have no offline unit tests because they require a live model; and there is no load test (Section 3.7).

9. Reflection

9.1 What went well

- Typed artifacts and a blackboard made every stage reviewable, re-runnable and auditable, and turned the ledger into a by-product rather than extra work.
- Putting structure in code — schemas, layout assembly, grounding — controlled the models far more reliably than prompt wording alone.
- Keeping the language model out of the veto made the Guardian cheap enough to run on every edit and testable in CI without network access.
- A real dispatch path forced the Guardian's checks to be ones that would genuinely stop a send, rather than demonstration checks.

9.2 What changed from the proposal, and why

Table 22. *Changes from the proposal*

Proposal	Build	Reason
Creative Director generates and critiques images in a loop	Written creative direction; image generation is an operator-triggered tool only	Per-segment images were costly, slow to review and hard to evaluate; a written direction is cheaper, reviewable and testable
Parallel creative ‖ copy fan-out	Strictly sequential graph	The direction became the composer's input, and each tab reviews the previous output
Per-recipient resolution as an agent stage	Merge-tag layer verified by the Guardian, filled at send	No per-recipient LLM call is needed; deterministic and cheaper
MJML compilation and accessibility linter	Builder-native `EmailBlock[]` with a single renderer	One renderer means the operator approves exactly what ships
Sandboxed dispatch stub	Real dispatch through Resend to a confirmed address	Makes the Guardian's checks consequential
Budget ceilings that halt a run	Cost meter for observability; segment cap and no loops	Simpler for a demonstrator; a hard ceiling is planned (R8)
Separate manipulation-index scorer	Prompt prohibitions + subject-line check + judge criterion	Comparable coverage with less machinery
Bounded REVISE loop back to the generator	Human-driven loop: edit or regenerate, then automatic re-check	Prioritised human review; the automated loop is the first next step

9.3 Challenges

- HDBSCAN needed C++ build tools on one Windows machine; containerising the backend on a slim image with prebuilt wheels removed the problem for deployment.
- Embedding support through OpenRouter was less standardised than chat; the embedding call is isolated in one function so the provider can be swapped without touching callers.
- Clean synthetic data clustered too neatly, which would have flattered segmentation; we added noise and overlap and accepted a lower, honest silhouette.

- Strict JSON-schema mode supports only a subset of JSON Schema; the client now rewrites schemas into that subset and keeps richer constraints in field descriptions and in code.
- On Windows, a shared HTTP/2 connection pool surfaced as socket error 10035 under overlapping requests; the client moved to HTTP/1.1 per thread.

9.4 Limitations and future work

1. Add a bounded Guardian → Composer revise loop (maximum three cycles) so REVISE failures such as subject-line or grounding issues are fixed automatically before a person sees them.
2. Add retry with exponential backoff and a fallback model to the OpenRouter client.
3. Add authentication with named approvers, per-route rate limits and a hard per-run budget ceiling; require a current-version verdict before send.
4. Replace the in-memory job registry with a shared store to allow multiple replicas, then load-test the pipeline at increasing list sizes.
5. Add an ingest-time injection scan and structural delimiting of free text; pin dependencies with hashes and add dependency auditing to CI.
6. Automate alerts on BLOCK-rate shifts and eval regressions, add LLM tracing, and calibrate the judge against a human-labelled sample.

9.5 Team contributions

Table 23. *Team contributions*

Member	Responsibilities
Ong Qing Quan (OQQ)	Engineering lead. Orchestrator and LangGraph graph, job runner and Decision Ledger; Segmentation & Insight agent; Email Composer & Assembly agent; Campaign Console including the email builder; OpenRouter client and cost meter; CI, versioning and Railway deployment.
Agustinus Benyamin Prasetyo (ABP)	Domain and assurance lead. Creative Direction agent; Compliance & Brand Guardian agent; synthetic data, brand and offer content; Responsible AI instrumentation including the proxy test and governance mapping; AI security test corpus.

Appendix A. API Surface

Group	Endpoints
State	`GET /api/state` · `GET /api/health` · `POST /api/reset` · `GET /api/jobs/{id}`
Intake	`GET` / `PUT /api/company` · `GET /api/audience` · `POST /api/audience/upload` · `POST /api/audience/load-sample`
Pipeline	`POST /api/segments/generate` · `GET /api/segments` · `POST /api/creative/generate` · `PATCH /api/creative/{id}` · `POST /api/emails/generate` · `PATCH /api/emails/{id}` · `POST /api/emails/{id}/adjudicate`
Gate and dispatch	`POST /api/approvals` · `DELETE /api/approvals/{id}` · `POST /api/emails/{id}/send` · `POST /api/email-assets/upload`
Evaluation and operations	`POST /api/eval` · `GET /api/logs` · `GET` / `PUT /api/settings`
Imagery (operator only)	`GET /api/images/config` · `POST /api/images/draft-prompt` · `POST /api/images/generate`

Appendix B. Repository Structure

Path	Contents
`backend/app/orchestrator/`	`graph.py` (LangGraph), `state.py` (typed blackboard), `agents/` (one file per agent)
`backend/app/eval/`	eval-v1 rubric, LLM judge, per-agent scorers, runner
`backend/app/routers/`	workspace, image_gen, test_send, misc
`backend/app/`	`ledger.py`, `openrouter_client.py`, `csv_ingest.py`, `brand.py`, `mailer.py`, `persistence.py`, `progress.py`, `cost_meter.py`, `config.py`
`backend/tests/`	LLMSecOps fixtures and Guardian tests; MLOps contract tests
`frontend/src/`	pages (one per tab), email builder components, `lib/pipeline.ts`
`supabase/migrations/`	`0001`–`0008`, applied in order
`.github/workflows/ci.yml`	The three CI jobs
`docs/`	Deployment guide, CI negative-test evidence, diagrams, this report and the presentation
`video/`	Remotion source for the 58-second intro video

Appendix C. Console Screenshots

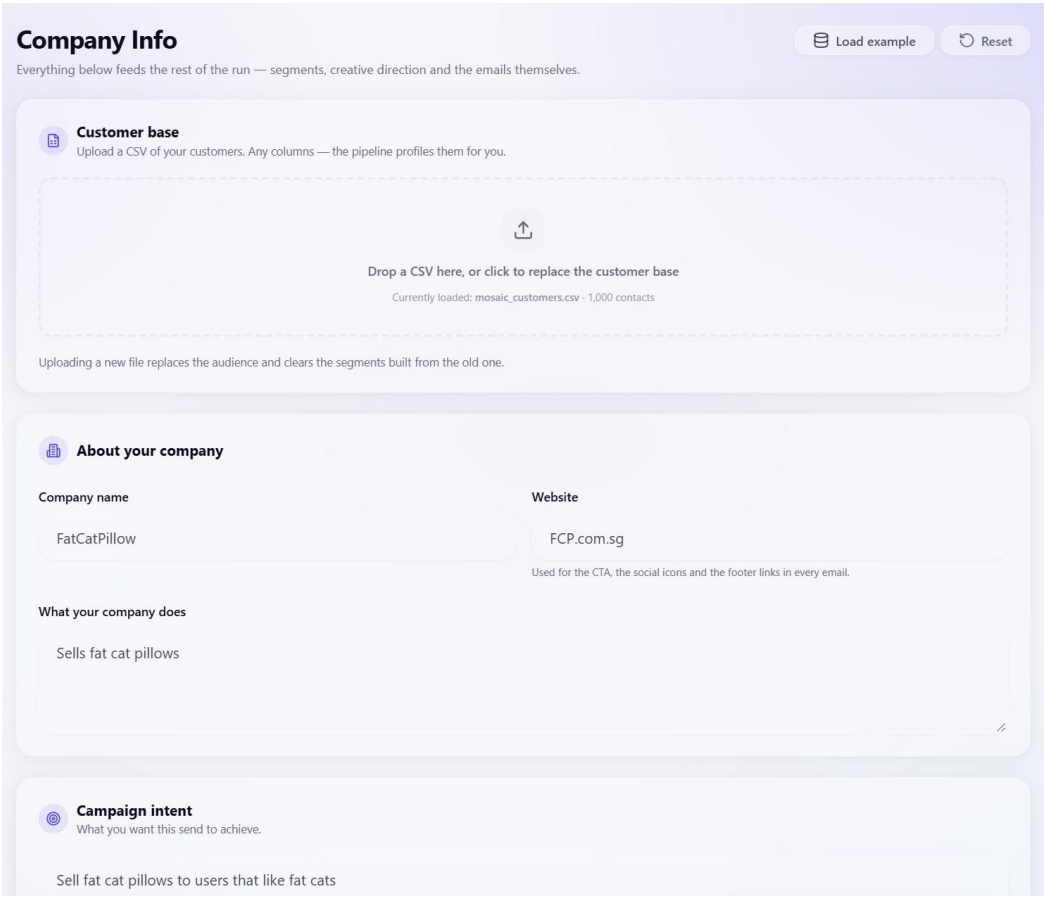


Figure 14. *Company Info: customer CSV upload, company profile and campaign intent.*

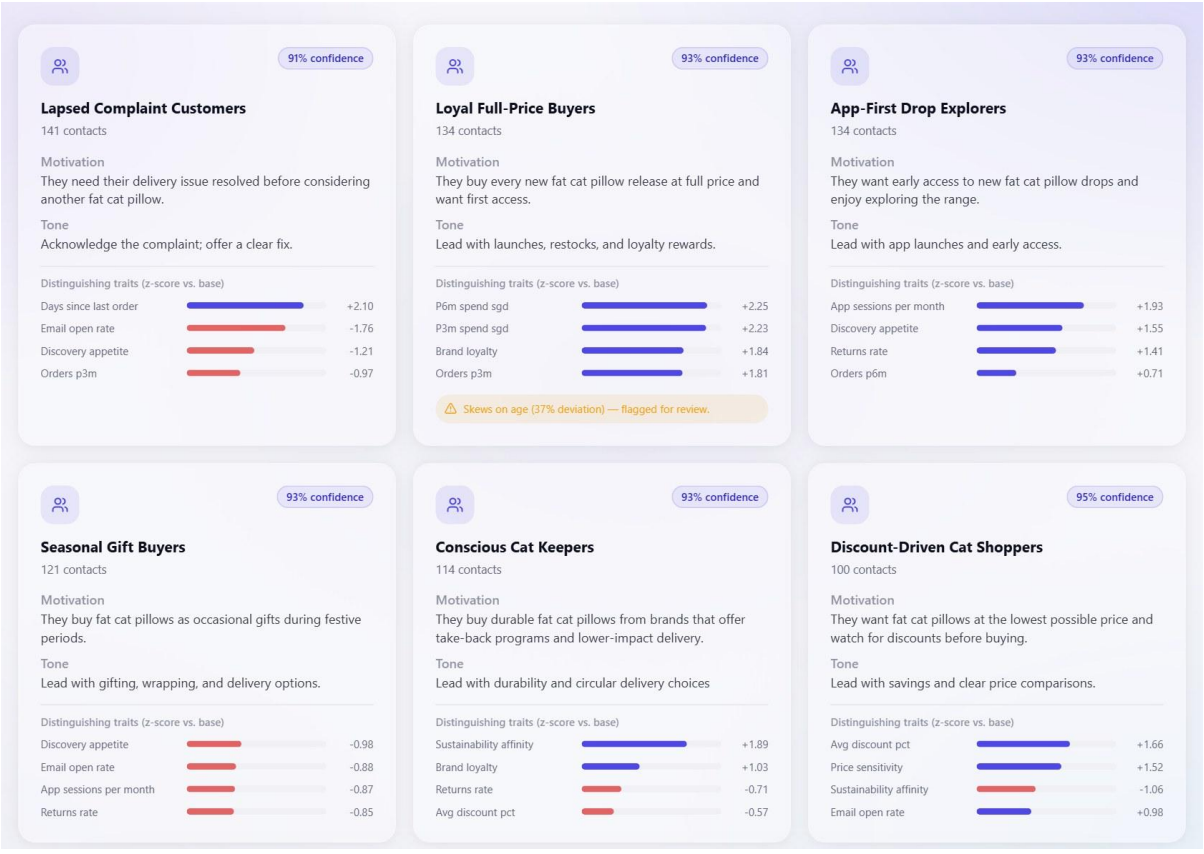


Figure 15. *Segments: six discovered segments with z-scored traits; the age-proxy flag appears on "Loyal Full-Price Buyers".*

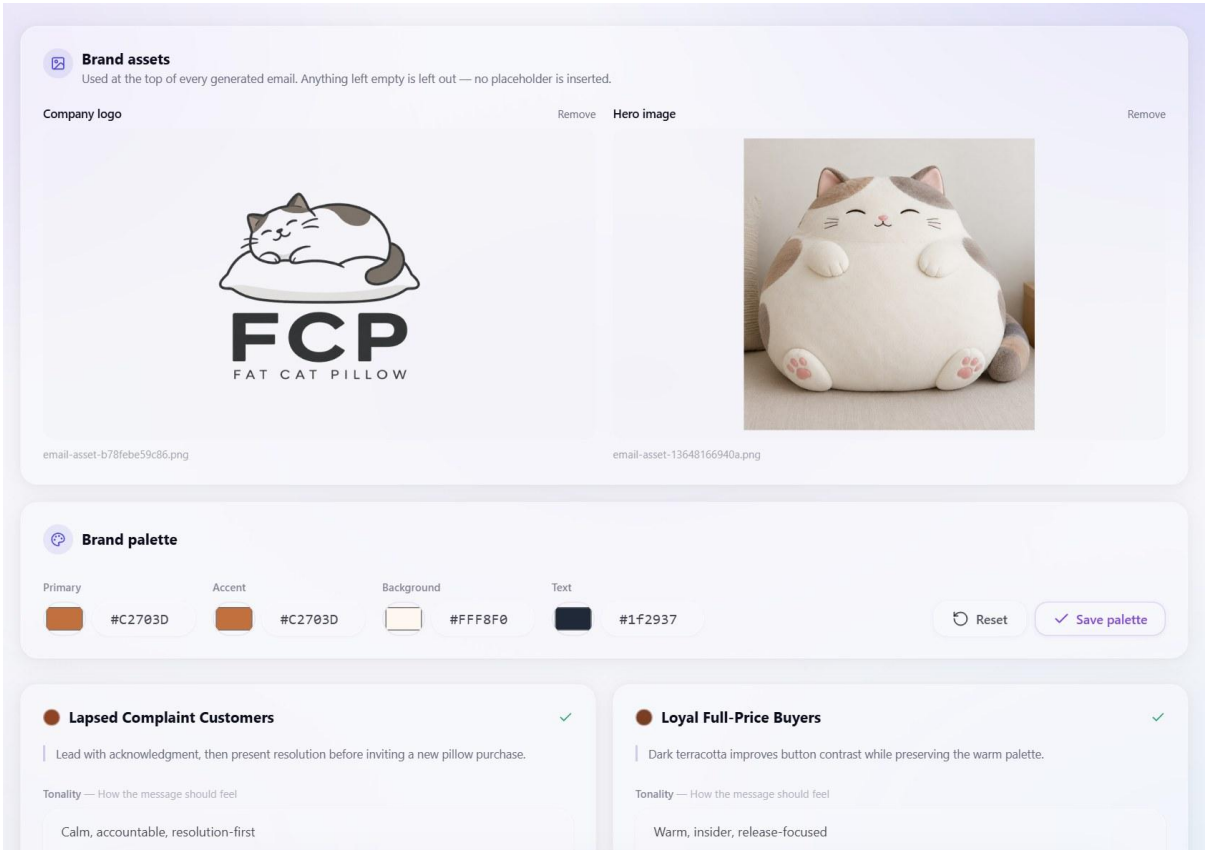


Figure 16. *Creative Review: brand assets, palette and per-segment direction.*

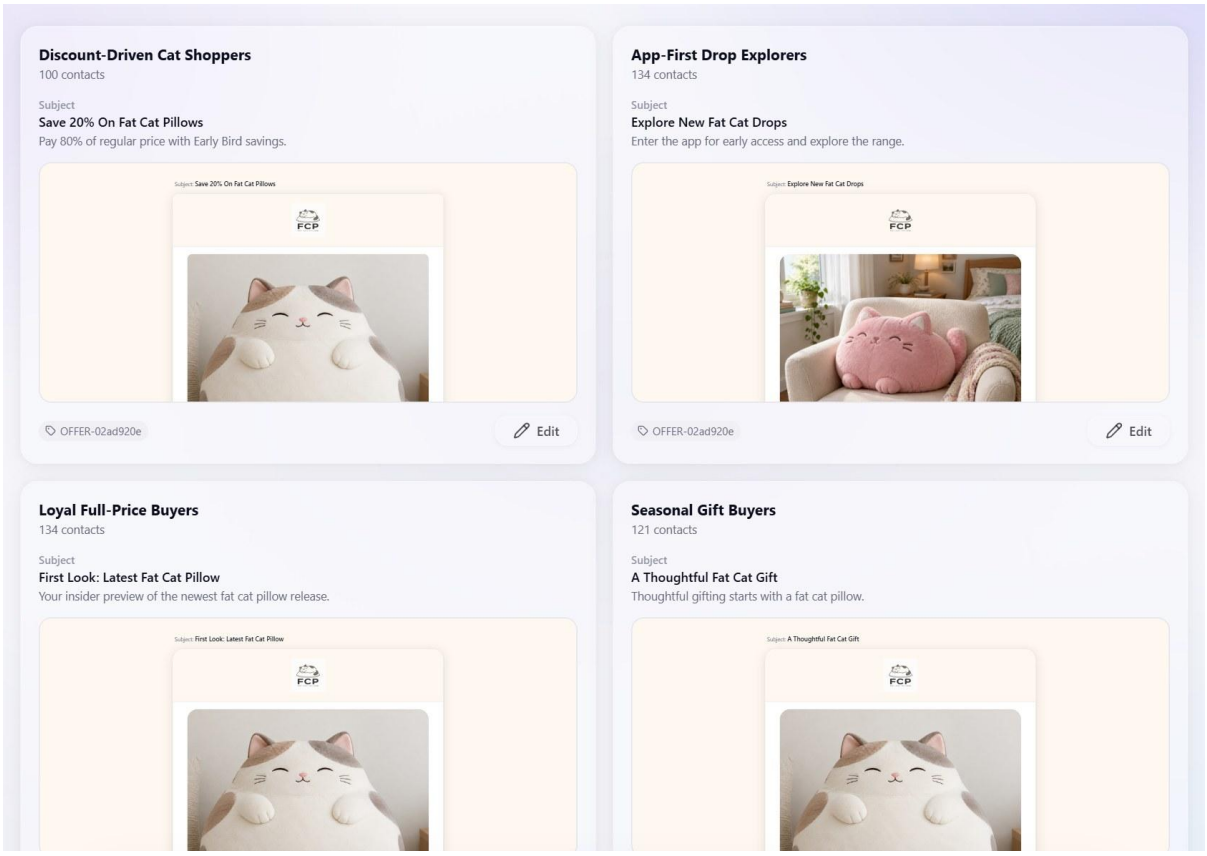


Figure 17. *Email Review: one composed email per segment, each opening in the drag-and-drop builder.*